

Read this before anything else

The PPT is not the source of truth. The repository is.

Nine things on the six submitted slides are either wrong, unbuilt, or contradicted by our own code. Every one of them is findable by a judge who reads carefully. Fix the slide, or be ready with the correction in your own mouth — a judge who finds it first has taken the room from you.

Verified against the repo on 2026-09-03. Cited by file, not by memory.

A0.1 [blocking] "A ML algorithm that analyzes the uploaded product image and description to suggest an optimal, competitive selling price" — slide 2

This is false three ways.

The slide says	The code does	Evidence
ML algorithm	Arithmetic. There is no model in the price path	<code>docs/decisions.md</code> : "Pricing is arithmetic, not a model"
Analyses the image	No image is sent anywhere in the price path. No image is sent to any model at all	<code>docs/app/AI-Data-Flow.md</code> , "Never sent" table
Analyses the description	Takes six numbers: material cost, labour hours, cluster, category, channel, MRP discount	<code>ai/price/compute.py</code> , <code>ai/contracts.md</code> POST <code>/price</code>

And the reason we did it this way is stronger than the slide. No training data exists for "what should this handicraft cost." A price predictor with no training set is a slide we cannot defend under one question. Cost-up arithmetic survives "*how did you get that number?*" — which on a government problem statement is the only question that matters.

The LLM appears exactly once in this feature (`ai/price/comps.py:20`) and **it never produces a price** — it normalises messy comparable listing titles so we compare like with like.

→ **Change the slide to:** "*Deterministic cost-up pricing with a loss-guard floor — every rupee is traceable to material cost, labour hours, and a cluster wage rate.*"

A0.2 [blocking] "Bhashini & LLMs for image enhancement" — slide 4, Technical Feasibility

Bhashini has nothing to do with image enhancement. It is MeitY's language mission — ASR, translation, TTS. Our image enhancement is BiRefNet, self-hosted, MIT licensed (`docs/decisions.md` #1, closed 2026-08-28 against a 5-model, 41-fixture benchmark).

Any judge who knows what Bhashini is will read this line as *"they do not know what their own stack does."* It is the cheapest credibility loss on the deck.

→ **Split it:** *"BiRefNet (self-hosted) for image matting; Bhashini for ASR/TTS across Indian languages."*

A0.3 [blocking] Node.js on slide 3 — we do not run Node in production

The flowchart's database block shows **Node.js + PostgreSQL**. The tech stack panel on the same slide shows **Python + FastAPI + PostgreSQL**. Both cannot be true.

Reality (`docs/decisions.md` #2, closed): **FastAPI + Postgres + Redis/RQ + S3-compatible object store**. Node exists only as the toolchain that builds the front ends (Vite, Next.js) — it serves no request.

→ **Delete the Node.js logo from the data layer.** Two conflicting stacks on one slide invites *"which one is it?"* and you lose thirty seconds you did not have.

A0.4 [open] "sarvam" is the only named AI vendor on slide 3 — and it appears nowhere in the code

The slide's "Voice, Marketplace & Cloud" panel shows Sarvam alone. What actually runs:

Job	What is wired today	File
Matting	BiRefNet, self-hosted, pinned revision	<code>ai/enhance/segmenter.py</code>
Voice → field value	OpenRouter, deepseek/deepseek-v4-flash (text only)	<code>ai/interpret.py</code>
ASR / TTS	Bhashini intended, terms unconfirmed. <code>docs/decisions.md</code> #5 is open	—
Description	Leaning Claude (<code>claude-sonnet-5</code>). <code>docs/decisions.md</code> #3	<code>ai/catalog/</code>

Job	What is wired today	File
	open	describe.py

Naming one vendor we do not use, while omitting the two we do, is a question we cannot answer without contradicting the slide.

→ **Either wire Sarvam and say so, or replace the logo with BiRefNet + Bhashini.** An Indian open-weights model is a genuinely good answer on this PS — but only if it is running.

A0.5 [open] "One Tap Listing to e-market places" — true for one channel, not six

Slide 2's USP panel shows Amazon, Flipkart, GeM, ONDC, Meesho and WhatsApp behind a single tap. What one tap actually does today (web/api/channels/registry.py:one_tap_channels):

Channel	Tier	Today	Honest word
Our marketplace	A	Real. Persisted and served	live
ONDC	A	Catalog mapped, <code>dry_run</code> — no subscriber id issued yet	mapped, not pushed
GeM	C	Renders <code>.xlsx</code> ; zero real category templates in the repo	file, when templates land
Amazon	B	OAuth real, payload real, <code>PUT</code> not implemented	connected, not pushed
Flipkart	B	Same shape	connected, not pushed
Meesho	D	Copy-paste block + guided browser	assisted

Our own registry docstring says it, in the code, unprompted:

"This said 'two live listings ... and it is real' while ONDC was returning a fabricated `ondc:{id}`. Put the two back when the push is real. The number here is the demo claim, and a demo claim that outruns the code is how you lose the room."

→ **Put the tier table on the slide.** "One tap, tiered — and here is the tiering" beats a flat claim that dies to the first industry judge. Tiering is a *stronger* answer, because it proves we know which platforms actually have APIs and which do not.

A0.6 [open] "Self-Improving AI Agent — Your Business Manager" — slide 2 USP

Half right, and the right half is better than the slide says. Corrected after reading `web/api/learning.py` properly — an earlier draft of this audit called it "tunable, not self-improving." **That was wrong.**

"Self-improving" is defensible. "Agent" is not.

Word	Verdict
Self-improving	[yes] Real, per artisan, and shipping. A closed feedback loop with a database behind it
Agent	[no] <code>docs/decisions.md</code> : " No agent framework for now. Every LLM call is one turn in, one JSON out. " No planning, no tool use, no multi-step autonomy

What actually runs — this is a genuine learning loop and it should be on the slide:

1. **FieldCorrection** (`web/api/models.py:231`) — a real table. Every time the artisan changes something we guessed, we store `{field, guessed, corrected, source}`.
2. **POST /catalog/corrections** (`routers/products.py:220`) writes it, fire-and-forget, so a lost row costs a little learning and never interrupts the artisan mid-correction.
3. **GET /catalog/defaults** reads their history *plus* their corrections and folds them through `learning.apply()`.
4. **learning.py decides what to do about being wrong** — and the rules are the interesting part: - **trusted()** — after 3 wrong guesses in the last 20 for a field, we **stop offering a guess for that field at all**. The comment says why the naive versions fail: one strike would let a single unusual product permanently silence a helpful field; ten strikes is *"ten confirmations they had to read, reject and re-record."* - **Scored per source** — the vision model being unreliable for this artisan says nothing about whether their own history is. *"Averaging them hides both."* - **A correction only counts when it changed something** — re-recording the same value because the microphone cut out is not evidence we were wrong. - **RECENT = 20** — a potter who moved from terracotta to stoneware corrected us a lot at the time; holding that forever *"means the feature degrades permanently for the artisans who use it most, which is exactly backwards."*
5. **CatalogVoice.tsx consumes it** — defaults pre-fill and skip questions, so **an artisan who has listed ten products is asked fewer questions than one listing their first**.

Two guardrails worth stating aloud, because they are what make it safe rather than clever:

- [warning] *"Nothing here ever writes a value into a listing on its own. The output is always 'offer this, as a confirmation' or 'offer nothing'. A default that publishes without the artisan confirming it is a guess under their name, and no amount of history earns that."*
- [privacy] **No prices are learnable.** `cost`, `price`, `mrp`, `floor_price` are absent from `CORRECTABLE_FIELDS` and must stay absent — *"last week's material spend under this week's photo is a wrong number in a place nobody would think to check."*

Backed by **test_learning.py, 145 lines**, testing the adversarial cases that made each naive rule wrong.

→ **Corrected slide wording:** *"Learns per artisan: the app remembers what it got wrong and stops asking questions it can already answer — and stops guessing entirely on any field it has been wrong about three times."* **Drop the word "agent".** Keep "self-improving" — it is earned.

The vector-database question, answered properly

No, and the reason is specific rather than a general preference.

The lookup is `WHERE artisan_id = ? AND field = ? ORDER BY created_at DESC LIMIT 20`. **Exact match on an indexed key, ordered by recency.** That is a B-tree. There is no similarity search anywhere in the path.

And a vector store would actively break `trusted().norm()`. `norm()` compares NFKC-casefolded exact strings *on purpose* — the whole rule counts **disagreements**. Under embedding similarity, "cotton" and "cotton blend" score as near-identical, so a genuine correction would stop counting as one, `wrong` would never reach 3, and **a field that keeps getting it wrong would keep confidently guessing forever.** The exactness is the feature.

Where a vector database genuinely will earn its keep, and this is the honest answer to give if pressed: **mapping a product to one of GeM's 10,700+ categories** (A0.5, and §8.4 of the Master Technical Reference). *That* is a semantic nearest-neighbour problem over a fixed taxonomy — the one place in this system where similarity search is the right tool. It is not built, and it is not what `learning.py` does.

A0.7 [open] "AI automatically removes cluttered backgrounds, corrects lighting, and enhances the photo" — slide 2

Background removal is real (BiRefNet, 402 ms warm on an RTX 4060). **Lighting correction is not written.** `CLAUDE.md`, Current state, verbatim:

"Three stages inside that sequence are still unwritten and are skipped explicitly, with every response naming them: `white_balance()`, `tone()`, `denoise_sharpen()`. Colour is the significant absence."

The pipeline is honest about it — every `/enhance` response names the skipped stages. **The slide is not.**

→ **Say what ships:** matting, tiering, crop-to-fill, pure-white composite, per-channel export, EXIF strip. Colour is next and we know exactly which function it goes in.

A0.8 [ok] Presentation hygiene — three free marks

Slide	Issue
-------	-------

2	Title is literally "IDEA TITLE" — the placeholder was never replaced
---	---

- | | |
|---|--|
| 1 | Team ID is blank |
| 6 | Header logo reads " SMART INDIA HACKATHON 2025 "; every other slide says 2026 |
| 6 | Key takeaway cites reference [6] ; the reference list ends at [5] |

None of these are technical. All four are visible in three seconds and all four say the same thing about care.

A0.9 [open] Numbers on slide 6 that need a primary source before they are spoken aloud

Slide 6 shows a "Digital readiness gap" bar chart (50% / 20% / 7% / 2%) and a "Barriers to online selling" pie (60% / 57% / 40% / 28% / 22.3%). The pie sums to 207.3%, so it is multi-select — say that out loud or it reads as an arithmetic error.

docs/Master-Technical-Reference.md §19 flags the artisan population figures as needing primary confirmation, and §18 item 11 assigns it. **The rule in our own repo is that nothing unverified goes on a judged slide.** These are on a judged slide.

Safe to state (primary-sourced): **GeM has 10,700+ product categories. IndiaHandmade onboarded 4,186 weavers and artisans between 2023–24 and June 2026** — against ~35 lakh handloom workers, ≈0.1% in three years. That second number is the entire thesis and it is defensible.

The rule this part exists to enforce

| *A claim that outruns the code is not ambition. It is the question you lose on.*

Every gap above has a true statement that is *more* impressive than the false one, because it proves we know where our own edges are. Nine corrections, zero loss of strength.